

A

Summer Internship Report

On

"CLOUDDASH – AUTONOMOUS MULTI-AGENT AI CUSTOMER SUPPORT SYSTEM"

(Summer Internship – II)

Prepared by

HET PATEL (23AIML051)

Under the Supervision of

Dr. Hardik Jayswal

Submitted to

Charotar University of Science & Technology (CHARUSAT)

for the Partial Fulfillment of the Requirements for the
Degree of Bachelor of Technology (B.Tech.)
in Artificial Intelligence and Machine Learning
for the 7th Semester

Submitted at



Accredited with Grade A+ by NAAC
Accredited with Grade A by KCG



Accredited with Grade A+ by NAAC
Accredited with Grade A by KCG



Chandubhai S. Patel Institute of Technology (CSPIT)
Faculty of Technology & Engineering (FTE), CHARUSAT
At: Changa, Dist: Anand, Pin: 388421.

July 2026



CERTIFICATE

This is to certify that the report entitled “**CLOUDDASH – AUTONOMOUS MULTI-AGENT AI CUSTOMER SUPPORT SYSTEM**” is a bonafide work carried out by **HET PATEL (23AIML051)** under the guidance and supervision of **Dr. Hardik Jayswal** and **Mr. Pallav Mamtora** for the **Summer Internship II (AIML-402)** of the 7th Semester of Bachelor of Technology in **Department of Artificial Intelligence and Machine Learning** at Chandubhai S. Patel Institute of Technology (CSPIT), Faculty of Technology & Engineering (FTE) – CHARUSAT, Gujarat.

To the best of our knowledge and belief, this work embodies the work of the candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred by the examiner(s).

Under the supervision of,

A handwritten signature in black ink, appearing to read "Hardik", is shown above a horizontal line.

Dr. Hardik Jayswal

Assistant Professor

Department of Artificial Intelligence and

Machine Learning

CSPIT, FTE, CHARUSAT, Changa, Gujarat

Mr. Pallav Mamtora

CEO

Techmicra IT Solutions, Ahmedabad

Dr. Nirav Bhatt

Head of Department (AIML)

CHARUSAT, Changa, Gujarat.

Chandubhai S. Patel Institute of Technology (CSPIT)
Faculty of Technology & Engineering (FTE), CHARUSAT
At: Changa, Ta. Petlad, Dist. Anand, Pin: 388421, Gujarat.



CERTIFICATE

There is to certify that **Het Patel (Enrollment No. 23AIML051) 6th Semester Chandubhai S Patel Institute of Technology** has successfully completed his internship at "Techmicra IT Solutions", Ahmedabad, during **16th May 2026 to 16th June 2026**

The Internship "**AI/ML**" was carried out as a part of academic requirement in **AI/ML for Chandubhai S Patel Institute of Technology**.

As a part of the internship till now he has completed the following tasks under our companies' guidance: -

Introduction to Artificial Intelligence (AI) and Machine Learning (ML), Python Programming for AI/ML, AI & ML Development Environment Setup, Data Preprocessing and Feature Engineering, Statistics and Mathematics for Machine Learning, Supervised and Unsupervised Learning, Machine Learning Algorithms, Deep Learning Fundamentals, Natural Language Processing (NLP), Computer Vision Basics, Model Training and Evaluation, Neural Networks and TensorFlow/PyTorch, Data Visualization, MLOps Fundamentals, Model Deployment and Monitoring, AI Ethics and Responsible AI, Git & GitHub for AI Projects, Cloud Platforms for AI/ML, Real-World AI/ML Applications, and AI/ML Project Development..

His understanding of problem context and technical knowledge for the tools used was up to the mark. During the internship work, we found him sincere and the work done by him was commendable.

We wish him all the best in his future endeavors and hope that he will have a successful career.

Sincerely, For,

For, **TECHMICRA IT SOLUTIONS**


Pallav Mamtara CEO
Techmicra IT Solutions



www.techmicra.in



info@techmicra.in | pallav@techmicra.in



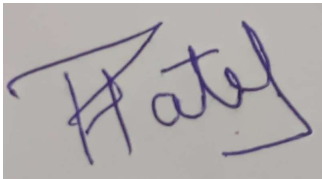
+91-9727835207 | +91-9427617574



DECLARATION BY THE CANDIDATE

I hereby declare that the internship report entitled "**CloudDash – Autonomous Multi-Agent AI Customer Support System**", submitted by **Het Patel** to Chandubhai S. Patel Institute of Technology (CSPIT), CHARUSAT, is a genuine record of the learning and project work undertaken during my summer internship at Techmicra IT Solutions, Ahmedabad.

I further declare that this report has been written based on the actual project implementation, the repository structure, execution logs, testing observations and the technical understanding I built up over the course of the internship. The work described here has not been submitted, in whole or in part, for the award of any other degree, diploma or academic credit at this or any other university.



Signature of the Candidate

HET PATEL
23AIML051

ACKNOWLEDGEMENT

I am sincerely grateful to Techmica IT Solutions, Ahmedabad, for giving me the chance to intern with them and to build the CloudDash project from the ground up. Working on a real multi-agent system, rather than a textbook exercise, taught me far more about applied AI engineering than I expected going in.

My thanks go to Mr. Pallav Mamtora, CEO of Techmica IT Solutions, for his direction and encouragement throughout the internship, and for pushing me to think carefully about how each agent in the pipeline should behave.

I would also like to thank my internal guide, Dr. Hardik Jayswal, Assistant Professor, Department of Artificial Intelligence and Machine Learning, CSPIT, CHARUSAT, for his continued academic supervision and for helping me connect the practical work back to the concepts covered in coursework.

I am grateful to the Department of Artificial Intelligence and Machine Learning at CSPIT, CHARUSAT, for the infrastructure and support that made this internship possible, and finally to my family and friends for their patience and encouragement over the past month.

I would also like to thank my classmates and batchmates who were working on their own internship projects in parallel during the same month — comparing notes on which retrieval strategies or prompting approaches were and weren't working turned out to be one of the more valuable parts of the whole experience.

I am also thankful to the maintainers of the open-source libraries and the documentation teams behind the APIs this project depends on — LangGraph, FastAPI, Pydantic, rank-bm25 and structlog among them — whose clear documentation made it possible to pick up several unfamiliar tools within the span of a single month.

Finally, I would like to acknowledge the broader CSPIT community for cultivating an environment where students are encouraged to take on internships of this depth rather than purely classroom-based projects, since it is that kind of applied, end-to-end experience that this report tries to document.

This internship has left me with a genuine interest in continuing to work on agentic AI systems beyond this report, and I am grateful to everyone named above for making that interest possible in the first place.

ABSTRACT

Most customer-facing chatbots still lean on a single language model to field every kind of question, which makes them shallow on specialised topics and hard to secure against misuse. This report describes CloudDash, an autonomous multi-agent customer support system built for a fictional cloud software company during a one-month AI/ML internship at Techmicra IT Solutions, Ahmedabad, carried out between 16 May 2026 and 16 June 2026.

Rather than routing every query to one generalist model, CloudDash splits the work across several cooperating agents orchestrated with LangGraph. An incoming message is first screened by a four-layer input guardrail that filters spam, jailbreak attempts and off-topic content. A triage agent then determines whether the query is technical or billing-related and detects the language it was written in, before a hybrid retriever searches a knowledge base of twenty articles using both dense vector search (NVIDIA NIM embeddings stored in Qdrant) and BM25 keyword search, merging the two rankings with Reciprocal Rank Fusion and reranking the result with an NVIDIA NIM reranker.

The retrieved passages are handed to a Technical Agent, a Billing Agent, or an Escalation Agent depending on the triage decision, each of which drafts its reply with the Groq LLaMA

3.3 70B model. A governed handover protocol lets these agents pass a conversation to one another, logging every transfer for audit purposes. Before any reply leaves the system, an output guardrail strips sensitive data such as card numbers and email addresses and checks that the answer is actually supported by the retrieved documentation rather than invented. The whole pipeline is exposed through a FastAPI backend with a lightweight browser chat interface for manual testing.

Manual testing across technical, billing and adversarial message types during the internship suggested that the guardrail-triage-retrieval-response structure holds up well for a small, well-defined product domain, while also making clear where a production version would need to invest further — principally in knowledge base coverage, latency, and persistent memory across sessions, all of which are discussed later in this report.

Contents

Company Certificate	iii
Declaration by the Candidate.....	iv
Acknowledgement	v
Abstract	vi
1 INTRODUCTION	
1.1 Background of Internship	1
1.2 Company Overview	1
1.3 Problem Statement	1
1.4 Objectives.....	1
1.5 Scope	2
1.6 Internship Timeline.....	2
2 REQUIREMENT ANALYSIS AND TECHNOLOGY STACK	
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	3
2.3 Technology Stack	4
3 SYSTEM ARCHITECTURE	
3.1 Pipeline Overview.....	5
3.2 Shared Conversation State.....	6
3.3 Agent Responsibilities.....	6
4 IMPLEMENTATION	
4.1 Input Guardrail Agent.....	7
4.2 Triage Agent	7
4.3 Hybrid Retrieval Agent.....	7
4.4 Specialist and Escalation Agents.....	7
4.5 Handover Protocol	8
4.6 Output Guardrail Agent	8
4.7 Frontend	8
4.8 API Endpoints	8
5 RESULTS AND TESTING	
5.1 Testing Approach.....	9
5.2 Sample Output	9
5.3 Technical findings.....	11
5.4 Observations.....	12
6 CONCLUSION AND FUTURE SCOPE	
6.1 Conclusion.....	13
6.1.1 Limitations	13
6.2 Future Scope.....	14
References	

CHAPTER 1

INTRODUCTION

1.1 Background of Internship

This report covers a one-month, project-based AI/ML internship completed at Techmicra IT Solutions, Ahmedabad, between 16 May 2026 and 16 June 2026, under the Summer Internship component of the 7th Semester B.Tech curriculum. Rather than moving between several small assignments, the internship centred on a single system, CloudDash, taken from an initial design sketch through to a working, end-to-end multi-agent application within the four-week window.

1.2 Company Overview

Techmicra IT Solutions is an Ahmedabad-based technology firm offering software development and IT services, with an active portfolio in AI/ML projects, web applications and enterprise solutions. The internship was run as a self-contained project: the intern was handed the full design-to-implementation responsibility for CloudDash and reviewed periodically by the company's CEO.

1.3 Problem Statement

A support chatbot built around a single general-purpose language model tends to run into the same set of problems in practice: it has no natural way to specialise between, say, a billing dispute and a server outage; it has no built-in defence against prompt injection or off-topic misuse; and it has no mechanism to check whether its own answer is actually grounded in real documentation before sending it to the customer. CloudDash was conceived to test whether splitting these responsibilities across a small team of purpose-built agents — coordinated through a graph-based orchestration framework rather than a single long prompt — produces a more reliable and auditable support pipeline.

1.4 Objectives

- Build a multi-agent pipeline in which distinct agents are responsible for distinct categories of customer problem, instead of one model trying to do everything.

- Combine dense vector search and keyword-based search into a single hybrid retrieval step so that agents answer from the knowledge base rather than from memory.
- Add input and output guardrails so the system resists misuse and does not leak sensitive data or unverified claims in its replies.
- Define a governed handover protocol so agents can transfer a conversation to one another in a controlled, logged way rather than silently.
- Expose the finished pipeline as a REST API with a basic browser chat interface for hands-on testing.

1.5 Scope

The internship covered the full build: the knowledge base and its ingestion pipeline, the retrieval logic, every agent in the graph, the two guardrails, the handover mechanism, the FastAPI service layer, and a minimal chat front end. Work was pushed to GitHub through the month following a day-by-day plan so that progress could be tracked commit by commit. Deployment to a production cloud environment and integration with a real company's CRM or ticketing system were treated as out of scope, although the architecture was kept flexible enough to support both later.

Similarly, load testing under concurrent traffic, formal security auditing of the guardrails, and multilingual expansion of the knowledge base itself were all identified early on as valuable next steps but were deliberately left outside the four-week scope, so that the core agent pipeline could be built and tested properly rather than spread thin across too many goals at once.

1.6 Internship Timeline

- Week 1 – Requirement study, knowledge base design (20 articles), and the four-layer input guardrail.
- Week 2 – Triage agent for intent and language detection, and the hybrid retrieval pipeline (BM25, NVIDIA NIM embeddings, RRF fusion, reranking).
- Week 3 – Technical, Billing and Escalation agents, the handover protocol, and the output guardrail.
- Week 4 – FastAPI integration, the browser chat interface, end-to-end testing, and report documentation.

CHAPTER 2

REQUIREMENT ANALYSIS AND TECHNOLOGY STACK

2.1 Functional Requirements

- Accept a customer message through a REST endpoint and screen it for spam, jailbreak attempts and off-topic content before any agent sees it.
- Determine whether an incoming query is technical, billing-related, or requires escalation, and detect the language it was written in.
- Retrieve the most relevant knowledge base passages for a query using a combination of semantic and keyword search.
- Generate a response grounded in the retrieved passages, in the same language the customer used.
- Allow a conversation to be transferred between agents under a defined set of allowed transitions, with every transfer logged.
- Redact sensitive information from outgoing replies and confirm that each reply is supported by retrieved documentation before it is released.
- Expose system health and handover history through dedicated API endpoints.

2.2 Non-Functional Requirements

- Reliability – the pipeline should degrade gracefully (e.g. explicitly stating that no documentation was found) rather than fabricating an answer when retrieval confidence is low.
- Security – the input guardrail should catch the great majority of prompt-injection and off-topic attempts before they reach a language model call.
- Auditability – every handover between agents must be traceable to a conversation ID, source agent, destination agent and timestamp.
- Maintainability – knowledge base articles should be addable without changing any agent code, only re-running the ingestion script.
- Responsiveness – a single customer message should resolve to a reply within a few seconds under normal API latency.

2.3 Technology Stack

- Python 3.11 as the implementation language, chosen for its mature AI/ML ecosystem and straightforward integration with the Groq and NVIDIA SDKs.
- LangGraph for defining the multi-agent graph and the conversation state passed between nodes, which made the deterministic, sequential pipeline much easier to reason about than a hand-rolled chain of function calls.
- FastAPI and uvicorn for the backend REST API and ASGI server, largely for the automatic request validation and the built-in interactive API documentation they provide.
- Pydantic v2 for validating requests and defining the conversation state schema, catching malformed input before it ever reached an agent.
- The Groq Python SDK for calling the LLaMA 3.3 70B and LLaMA 3.1 8B models, selected mainly for their low inference latency compared to the alternatives tried during the first week.
- The OpenAI Python SDK, used as a compatibility client for the NVIDIA NIM embedding and reranker endpoints, since NIM exposes an OpenAI-compatible interface.
- The Qdrant client for storing and querying article embeddings in Qdrant Cloud, chosen for its straightforward Python client and generous free tier for a project of this size.
- rank-bm25 for the keyword-search half of the hybrid retriever, a lightweight library that avoided pulling in a full search engine for just twenty documents.
- structlog for structured JSON logging across the pipeline, which made it far easier to trace a single conversation through every agent when something went wrong during testing.
- Visual Studio Code, Git/GitHub, Postman and PowerShell as the day-to-day development toolchain.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Pipeline Overview

CloudDash is organised as a deterministic, sequential graph rather than a loose collection of independent calls. Every customer message travels through the same fixed sequence of nodes

— input guardrail, triage, hybrid retrieval, a specialist agent, and output guardrail — with LangGraph managing the transitions between them and the state that accumulates along the way. Figure 3.1 shows this flow.

This determinism was a deliberate trade-off. A more flexible graph, where the model itself decides which agent to call next, was considered during the design phase, but a fixed sequence made the system’s behaviour far easier to predict, test and explain — useful properties for anything customer-facing, and especially useful when trying to debug a specific failing conversation during a four-week internship.

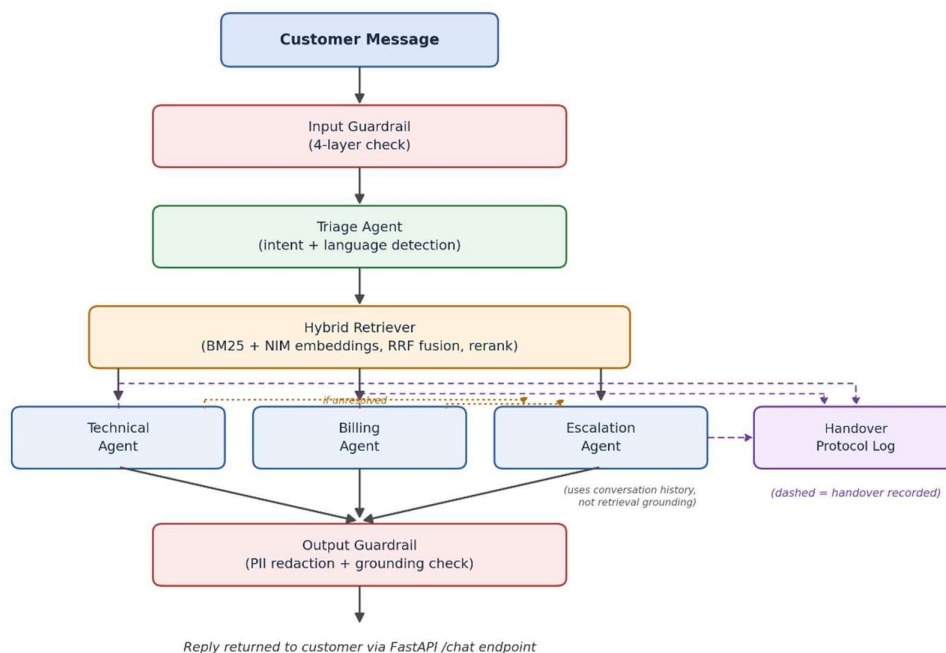


Figure 3.1: Multi-agent pipeline architecture of CloudDash.

3.2 Shared Conversation State

A single state object, defined as a Pydantic model, is threaded through every node in the graph. It starts out holding just the raw customer message, and each agent reads what it needs and writes back what it produces: the guardrail nodes add pass/fail flags, the triage agent adds the detected intent and language, the retriever adds the ranked passages it selected, and the specialist agent adds its drafted reply. Because the state is shared rather than re-derived at each step, later agents never lose context that an earlier agent already worked out.

This shared-state pattern also made debugging considerably easier during development: since the entire state object could be logged at any node, tracing a conversation that produced an unexpected reply was mostly a matter of reading back through the state snapshots to see exactly where the triage decision, retrieval result, or guardrail flag diverged from what was expected.

3.3 Agent Responsibilities

- Input Guardrail – rejects unsafe or off-topic messages before any language model is invoked, keeping the more expensive downstream calls reserved for genuine queries.
- Triage Agent – classifies intent (technical, billing, escalation) and detects language, setting the routing decision that every later node relies on.
- Hybrid Retriever – rewrites the query and fetches the most relevant knowledge base passages using both dense and keyword search.
- Technical Agent / Billing Agent – draft a grounded reply for their respective domain, using only the passages the retriever supplied.
- Escalation Agent – packages the conversation for human hand-off when the specialist agents cannot resolve it, preserving the full message history for the human agent who picks it up.
- Output Guardrail – redacts sensitive data and verifies the reply is grounded before release, acting as the last checkpoint before anything reaches the customer.

IMPLEMENTATION

4.1 Input Guardrail Agent

Every message is screened by four checks run in sequence before it reaches any agent. The first uses a set of nineteen regex patterns to catch known jailbreak and prompt-injection phrasing. The second scans for known CloudDash product terms and lets the message through immediately if any are found, avoiding an unnecessary model call for obviously on-topic queries. The third applies hard-block regex rules to reject content that is clearly unrelated to the product. Anything that survives the first three checks but is still ambiguous is finally classified by the Groq LLaMA 3.1 8B model, which decides whether the message should proceed.

4.2 Triage Agent

Once a message clears the guardrail, the Triage Agent — running on Groq LLaMA 3.3 70B — reads it and decides two things: which category of problem it represents (technical, billing, or escalation) and which language it is written in. Both values are written into the shared state so that every downstream node, from the retriever to the specialist agent, can use them without re-deriving them.

4.3 Hybrid Retrieval Agent

Before any specialist agent is allowed to answer, the Hybrid Retriever pulls the most relevant knowledge base passages. It first rewrites the raw customer query with Groq LLaMA 3.1 8B to make it more search-friendly, then runs two searches in parallel over the same twenty knowledge base articles: a dense vector search using NVIDIA NIM embeddings stored in Qdrant, and a BM25 keyword search using rank-bm25. The two ranked lists are merged with Reciprocal Rank Fusion ($k = 60$) and the merged result is reranked by an NVIDIA NIM reranker model to surface the strongest passages. If the top similarity score still falls below 0.5, the agent is instructed to say plainly that no matching documentation was found rather than guess.

4.4 Specialist and Escalation Agents

The Technical Agent handles dashboard errors, outages, API problems and monitoring alerts, while the Billing Agent covers invoices, payments, refunds and subscription changes; both receive the retrieved passages as grounding context and generate their reply with Groq LLaMA3.3 70B in the customer's detected language. When neither specialist can resolve the issue, the Escalation Agent assembles the full conversation history and flags it as requiring a human agent.

4.5 Handover Protocol

Agents are not free to hand a conversation to any other agent — the allowed transitions are defined in a configuration file, and any attempt outside that list is rejected. Every accepted handover is written to a JSONL log with the conversation ID, the source and destination agents, and a timestamp, giving a complete audit trail of how a conversation moved through the system.

4.6 Output Guardrail Agent

Before a reply is released to the customer, the Output Guardrail runs two checks. It applies regex patterns to find and strip credit card numbers, social security numbers and email addresses, and it performs a grounding check that compares the drafted reply against the retrieved knowledge base passages to catch hallucinated claims. A reply is only sent once it clears both checks.

The grounding check proved to be the more interesting of the two in practice: rather than a simple keyword overlap test, it asks a language model to judge whether each factual claim in the drafted reply is actually supported by the retrieved passages, which catches subtler cases

— such as a reply that quietly generalises beyond what the documentation actually says — that a keyword-matching approach would have missed.

4.7 Frontend

A lightweight, browser-based HTML chat interface was built to exercise the pipeline manually. It sends each typed message to the /chat endpoint, displays the returned reply, and also shows which agent ultimately produced it — useful for confirming that triage and handover decisions were being made correctly during testing.

4.8 API Endpoints

The backend exposes three FastAPI endpoints:

- /chat – accepts a customer message and returns the agent-generated reply along with routing metadata.
- /health – reports system status, the number of knowledge base articles loaded, and the Qdrant connection state.
- /handovers – returns the contents of the handover audit log for inspection.

CHAPTER 5

RESULTS AND TESTING

5.1 Testing Approach

The pipeline was tested manually through the browser chat interface and Postman, using a mix of message types: single-intent technical queries, deliberately off-topic messages, and messages that shifted context mid-conversation (starting technical, then turning into an urgent billing complaint). The chat interface also exposes a set of preset quick-reply buttons (“Alert issue,” “SSO + Upgrade,” “Double charge,” “KB miss”) that were used alongside manually typed messages to exercise common scenarios repeatably. Each response in the interface is labelled with the agent that produced it (TRIAGE, TECHNICAL, ESCALATION) and a trace ID, which made it straightforward to confirm which stage of the pipeline had handled a given message. The goal was to confirm that each stage of the pipeline behaved as designed rather than to run a formal, automated test suite.

5.2 Sample Output

An off-topic message — “Hello today is which date” — was correctly identified as unrelated to CloudDash. The system responded under the TRIAGE label: “I can only assist with CloudDash support topics such as technical issues, billing, account management, and integrations. How can I help you with CloudDash today?” (Figure 5.1).

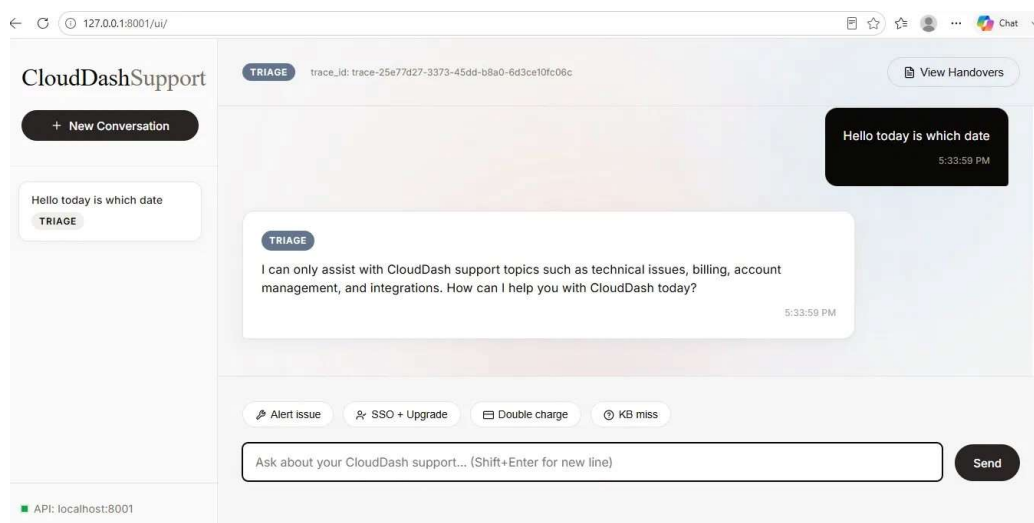


Figure 5.1: Off-topic message correctly declined by the Triage Agent.

A technical query — “My CloudDash alerts stopped firing after I updated my AWS integration credentials yesterday. I’m on the Pro plan.” — was routed to the Technical Agent, which cited knowledge base article KB-005 and gave a specific, step-by-step fix: forcing a credential refresh via Settings → Integrations → AWS → Re-validate Credentials, waiting for the 2–3 minute cache to clear, checking the integration health panel for error codes such as AUTH_FAILED or PERMISSION_DENIED, and manually testing the alert to isolate the fault

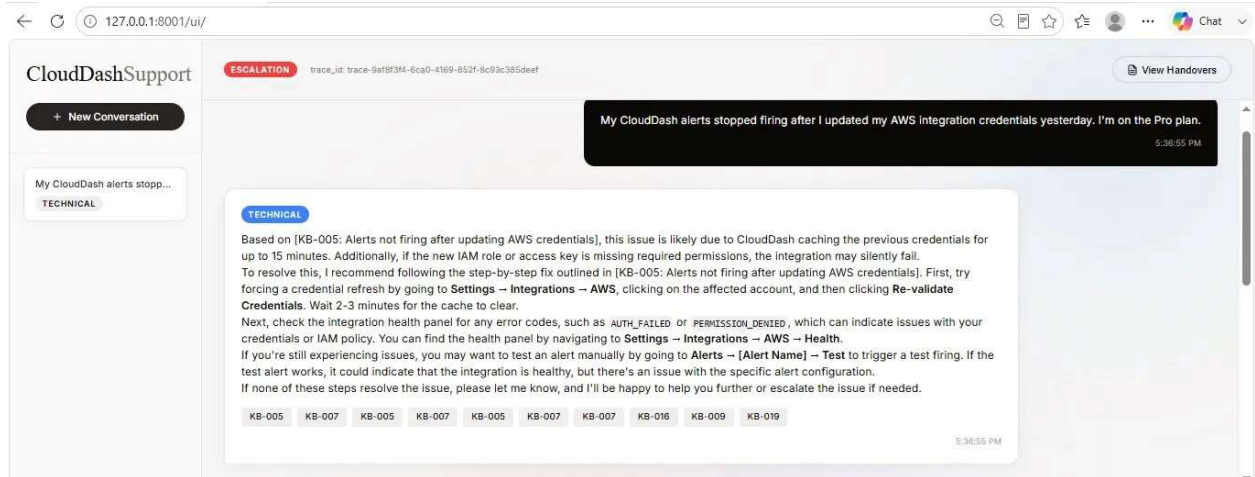


Figure 5.2: Technical Agent resolving an AWS integration issue with cited KB articles.

When the same conversation continued with a follow-up message — “I’ve been charged twice for April. I need an immediate refund and I want to speak to a manager.” — the system did not route this to a Billing Agent. Instead, it handed the conversation over from the Technical Agent to the Escalation Agent, logging the reason directly in the interface as “Agent determined context transition needed.” The Escalation Agent opened escalation case CD-06484978 and ticket ESC-428, set the priority to high, assigned it to the technical-tier2 team, and confirmed a first response within two hours (and human contact within four). The conversation input was then locked, with the interface displaying “Escalated — conversation locked” (Figure 5.3).

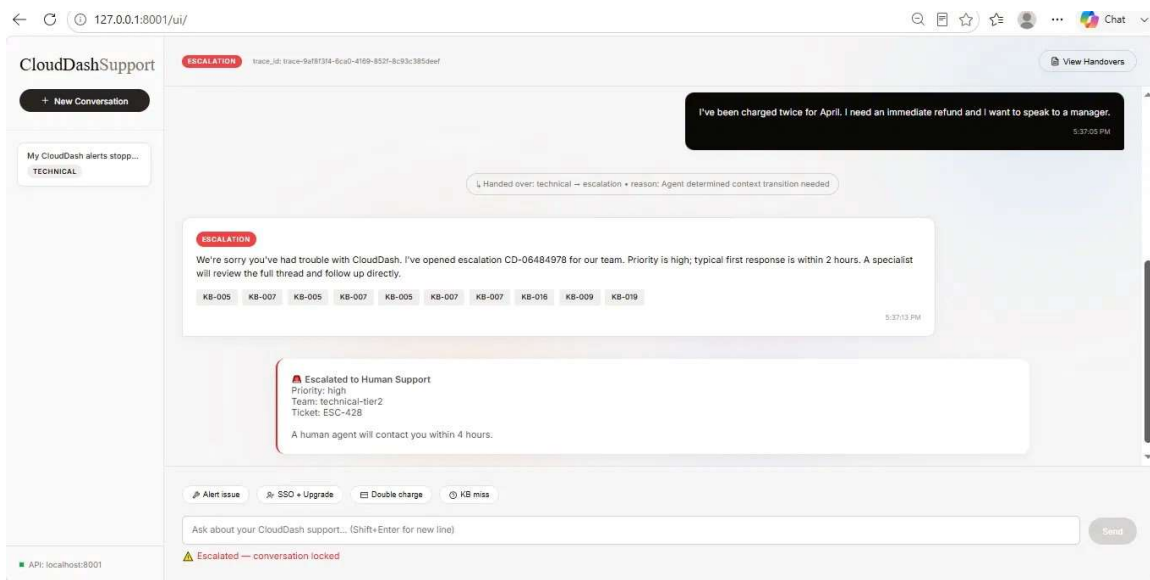


Figure 5.3: Conversation handed over to the Escalation Agent and locked after ticket creation.

5.3 Technical Findings from Execution Evidence

- The guardrail correctly distinguished a conversationally-phrased but genuinely off-topic message (“Hello today is which date”) from a genuine support query, rather than relying only on obvious keyword mismatches.
- When a customer’s message contained explicit escalation language — asking for an “immediate refund” and “to speak to a manager” — the handover protocol routed the conversation to the Escalation Agent rather than to the Billing Agent, treating the customer’s request for human intervention as taking priority over routine billing handling.
- The handover reason, the destination agent, and the resulting ticket details (priority, team, ticket ID, response-time commitment) were all surfaced directly in the customer-facing interface rather than only in the backend log, which is a stronger audit trail than the log alone would provide.
- Once a conversation was escalated, the interface locked further input, preventing the automated agents from replying again after responsibility had been handed to a human— consistent with the handover protocol’s intent of a controlled, one-way transfer.

5.4 Observations

Rather than running two specialist agents in parallel when a conversation touches more than one topic, CloudDash’s actual behaviour is to hand the whole conversation over to a single downstream agent — in the tested case, from Technical to Escalation — once the customer’s message signals that the current agent is no longer the right one to continue. This is a more conservative design than attempting to split a mixed-intent message across two specialist agents at once, and it matches the handover protocol described in Chapter 4: a conversation is owned by one agent at a time, and the transfer between agents is explicit and logged rather than implicit.

One consequence worth noting is that a message combining a resolvable technical detail with an urgent billing complaint was escalated to a human in its entirety, rather than the technical part being closed out and only the billing part being escalated. For a customer whose technical issue was already answered adequately by the Technical Agent, this means a straightforward billing question can still end up needing a human, simply because it was raised in the same breath as a demand to speak to a manager.

It was also noticeable that the quality and usefulness of a reply depended heavily on how specific the customer’s message was — the AWS credentials example, which named the exact integration and plan tier, produced a precise, directly actionable fix, while vaguer phrasing would likely rely more heavily on the retriever’s ranking to guess at relevant context.

Taken together, these observations suggest the escalation-first behaviour for urgent or dissatisfied-sounding messages is a reasonable, conservative default for a customer support system, but leaves room for a future refinement: distinguishing a customer who is frustrated but still has a resolvable technical or billing question from one who genuinely needs a human, rather than escalating .

CHAPTER 6 CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This internship set out to test whether a customer support workflow could be handled more reliably by a coordinated team of specialised agents than by a single general-purpose chatbot, and the working CloudDash system built over the four weeks supports that idea. Splitting the pipeline into an input guardrail, a triage agent, a hybrid retriever, specialist response agents and an output guardrail made each stage easy to reason about and test in isolation, while LangGraph’s shared conversation state kept them working from the same picture of the ongoing exchange.

Along the way, the internship gave hands-on experience with a genuinely production-oriented toolchain — LangGraph for orchestration, FastAPI and Pydantic for the service layer, Groq for fast LLM inference, and Qdrant with NVIDIA NIM for hybrid retrieval — as well as practical exposure to concerns that rarely come up in coursework, such as prompt-injection defence, output grounding checks and PII redaction.

Perhaps the most useful takeaway was a more concrete sense of where a multi-agent system actually needs careful engineering attention: not so much in getting a language model to produce a plausible-sounding reply, which turned out to be the easy part, but in getting the surrounding scaffolding — retrieval, routing, guardrails and logging — solid enough that the system's behaviour stays predictable once real, messy customer input starts arriving.

6.1.1 Limitations

- The knowledge base holds only twenty articles, well short of what a real support product would need, and adding new ones currently requires re-running the ingestion script by hand.
- The system depends on three external services — Groq, NVIDIA NIM and Qdrant Cloud — and cannot function if any of them is unavailable; each message also triggers several chained API calls, which adds both latency and cost.
- Knowledge base articles are written only in English, so while an agent can reply in the customer’s detected language, the retrieved context behind that reply is still English-only for non-English speakers.
- There is no persistent memory across sessions, so a returning customer’s earlier conversation is not available to the agents on a later visit.

- The system runs locally without authentication, rate limiting or load balancing, so it is not yet ready for a production deployment as it stands.
- Testing during the internship was manual rather than automated, so there is no regression suite yet to catch a change to one agent's prompt accidentally breaking behaviour elsewhere in the pipeline.
- The system does not split a mixed-intent conversation across two specialist agents; once a message signals a context shift — such as a technical conversation turning into an urgent billing complaint — the entire conversation is handed to the Escalation Agent, even if part of the original issue was already resolved.

6.2 Future Scope

- Expand the knowledge base to cover the full range of product features, error codes, billing policies and SLA terms, and build an admin interface so support staff can maintain it without touching the ingestion script.
- Add persistent conversation memory so agents can recall a customer's history and personalise responses across sessions.
- Translate the knowledge base into multiple languages so retrieved context, not just the final reply, matches the customer's language.
- Deploy the FastAPI backend to a cloud platform with authentication, rate limiting and load balancing, backed by a CI/CD pipeline for automated testing and releases.
- Build an analytics dashboard tracking conversation volume, common intents, average response time and escalation rate to guide future improvements to the knowledge base.
- Introduce automated regression tests covering the guardrails and the retrieval pipeline, so future changes to prompts or models can be validated against a fixed set of known-good and known-bad conversations before release.
- Allow the triage decision to be revisited mid-conversation rather than fixed at the first message, so a conversation that starts ambiguous can still be re-routed correctly once the customer clarifies their issue.

References

- [1] LangChain AI, “LangGraph Documentation”, <https://langchain-ai.github.io/langgraph/>
- [2] FastAPI, “FastAPI Documentation”, <https://fastapi.tiangolo.com/>
- [3] Groq, “Groq API Documentation”, <https://console.groq.com/docs/>
- [4] NVIDIA, “NVIDIA NIM API Documentation”, <https://build.nvidia.com/>
- [5] Qdrant, “Qdrant Vector Database Documentation”, <https://qdrant.tech/documentation/>
- [6] Pydantic, “Pydantic v2 Documentation”, <https://docs.pydantic.dev/>
- [7] Cormack, G.V., Clarke, C.L.A., and Buettcher, S., “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”, SIGIR 2009.
- [8] OWASP, “OWASP Top 10 for Large Language Model Applications”, <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [9] Robertson, S. and Zaragoza, H., “The Probabilistic Relevance Framework: BM25 and Beyond”, Foundations and Trends in Information Retrieval, 2009.
- [10] structlog, “structlog Documentation”, <https://www.structlog.org/>